ОТЧЕТ

Node Exporter, Prometheus, Grafana

Выполнил(а): Барыктабасов Бекзат
Группа: БИС-1-25
Проверил(а): Жалилов Г. Э.

Бишкек 2026 г.

ОТЧЁТ О ПРОДЕЛАННОЙ РАБОТЕ

Проект: Bank Infrastructure Monitoring

Тема: Создание системы мониторинга серверов с помощью Prometheus, Grafana, Node Exporter с отправкой уведомлений в Telegram.

Цель: Научиться следить за состоянием нескольких серверов, замечать проблемы с их работой и получать автоматические уведомления, когда что-то перестаёт работать или возникает высокая нагрузка.

Среда: VMware Workstation, Ubuntu Server.

1. С чего всё началось

Я начал этот проект после того, как получил задание по мониторингу серверов в рамках отбора на стажировку в Бакай Банк.

Нужно было установить Node Exporter, Prometheus и Grafana, самостоятельно создать дашборд и настроить уведомления в Telegram.

До начала работы я понимал, что эти инструменты используются для мониторинга, но у меня ещё не было опыта создания такой системы целиком, с несколькими серверами, отдельной базой данных и проверкой настоящих срабатываний алертов.

Поэтому я решил не ограничиваться установкой программ. Мне хотелось построить небольшую инфраструктуру, в которой несколько серверов работают вместе, а один центральный сервер собирает информацию об их состоянии.

Для этого я использовал гипервизор VMware Workstation. Все серверы были виртуальными, то есть работали на моём компьютере.

Дополнительно, также я решил добавить PostgreSQL, чтобы научиться следить не только за процессором, оперативной памятью и диском, но и за работой самой базы данных.

2. Создание трёх виртуальных серверов

Первым делом я создал три виртуальные машины с Ubuntu Server.

Название	IP-адрес	Для чего использовал
MON-01	192.168.56.10	Сервер, на котором работают Prometheus и Grafana
APP-01	192.168.56.20	Сервер для проверки мониторинга и создания тестовой нагрузки
DB-01	192.168.56.30	Сервер с PostgreSQL

MON-01 стал главным сервером мониторинга. Именно на нём я разместил Prometheus, который собирает данные, и Grafana, которая показывает их в виде графиков.

APP-01 я использовал как отдельный сервер, за которым можно наблюдать. Например, на нём можно повысить загрузку процессора или остановить Node Exporter и посмотреть, как система мониторинга отреагирует на это.

DB-01 я выделил под базу данных PostgreSQL. Позже на нём установил PostgreSQL Exporter, чтобы собирать информацию о работе базы.

Я понял, что разделение серверов помогает лучше разобраться в том, как устроен мониторинг. Когда все программы установлены на одном компьютере, некоторые вещи можно не заметить. А когда серверы разные, приходится правильно настраивать сеть и проверять, может ли один сервер подключиться к другому.

3. Настройка сети между виртуальными машинами

После создания серверов мне нужно было сделать так, чтобы они могли общаться между собой.

Для этого я использовал два сетевых адаптера VMware.

3.1. NAT — доступ к интернету

Первый адаптер работал в режиме NAT.

Я использовал сеть:

192.168.201.0/24

Она была нужна для того, чтобы виртуальные серверы могли выходить в интернет.

Например, когда я выполнял:

sudo apt update

Ubuntu обращалась к серверам с пакетами и получала информацию об обновлениях.

А когда я выполнял:

sudo apt install prometheus

система могла скачать и установить Prometheus.

3.2. Host-only — внутренняя сеть

Второй адаптер я настроил в режиме Host-only.

Для него использовал сеть:

192.168.56.0/24

Именно в этой сети находились мои три сервера.

MON-01 получил адрес 192.168.56.10, APP-01 — 192.168.56.20, а DB-01 — 192.168.56.30.

Host-only позволил создать отдельную сеть между моим компьютером и виртуальными машинами. Эта сеть не была напрямую доступна из интернета.

Я выбрал такую схему, потому что мне нужно было, чтобы Prometheus обращался к другим серверам по внутренним IP-адресам.

3.3. Проблемы с сетью

Во время настройки у меня не всё получилось с первого раза.

Были проблемы с доступностью виртуальных машин. Пришлось разбираться с виртуальными сетевыми адаптерами, настройками VMware и IP-адресами.

Также обнаружился конфликт с сетевым адаптером VirtualBox на Windows.

Я проверял сетевые настройки и исправлял конфигурацию, пока серверы не начали нормально взаимодействовать.

Этот этап оказался полезным, потому что я понял, насколько важна правильно настроенная сеть. Даже если Prometheus и Node Exporter установлены без ошибок, мониторинг не будет работать, если между серверами нет сетевого соединения.

4. Настройка удалённого подключения через SSH

После настройки сети я начал подключаться к виртуальным серверам через SSH.

До этого для управления виртуальной машиной можно было открывать её окно в VMware и вводить команды там.

Но когда серверов несколько, такой способ не очень удобен.

Поэтому я использовал PowerShell на Windows и подключался к Ubuntu через SSH.

Например:

ssh lucgok@192.168.56.10

Эта команда подключала меня к MON-01.

После подключения я мог выполнять команды Linux прямо из PowerShell.

Через SSH я устанавливал программы, проверял службы, открывал конфигурационные файлы и перезапускал компоненты мониторинга.

5. Установка Node Exporter на все серверы

Когда сеть заработала, я перешёл к установке Node Exporter.

Сначала я разобрался, для чего он вообще нужен.

Node Exporter — это программа, которая получает информацию о состоянии Linux-сервера и предоставляет её в виде метрик.

Например, с её помощью можно узнать, насколько загружен процессор, сколько используется оперативной памяти, сколько свободного места осталось на диске и какой объём сетевого трафика проходит через сервер.

Я установил Node Exporter на все три виртуальные машины.

Для установки использовал команду:

sudo apt install prometheus-node-exporter -y

После установки проверял состояние службы:

sudo systemctl status prometheus-node-exporter

В результате у меня появились три адреса для получения метрик:

192.168.56.10:9100

192.168.56.20:9100

192.168.56.30:9100

Я проверял адрес /metrics, через который Node Exporter отдаёт информацию.

Сначала там было очень много непонятных строк с названиями метрик и числами. Позже, когда я начал писать PromQL-запросы, стало понятнее, что именно означают эти данные.

Что я понял о Node Exporter

Самое важное, что я узнал: Node Exporter не отправляет данные в Grafana и не отправляет их самостоятельно в Prometheus.

Он просто предоставляет метрики по HTTP.

А Prometheus сам обращается к Node Exporter через определённые интервалы и забирает информацию.

То есть Node Exporter отвечает на запрос, а Prometheus выполняет сбор данных.

Такой способ работы называется Pull Model.

Также я понял, что порт 9100 не стоит открывать в интернет без защиты, потому что метрики могут раскрывать информацию о сервере.

6. Установка и настройка Prometheus

После Node Exporter я установил Prometheus на MON-01.

Prometheus нужен для централизованного сбора и хранения метрик.

Если Node Exporter только предоставляет данные, то Prometheus регулярно собирает их и сохраняет, чтобы потом можно было смотреть текущие и прошлые значения.

Для установки я использовал:

sudo apt install prometheus -y

В моей лаборатории использовался Prometheus версии 2.53.5.

После установки я открыл веб-интерфейс:

http://192.168.56.10:9090

6.1. Подключение трёх серверов

Чтобы Prometheus начал собирать данные с моих виртуальных машин, мне нужно было указать их адреса в конфигурационном файле.

Я работал с файлом:

/etc/prometheus/prometheus.yml

В нём есть раздел scrape_configs.

В этот раздел я добавил адреса Node Exporter трёх серверов.

Пример конфигурации:

```
- job_name: 'node_exporter'
  static_configs:
    - targets:
        - '192.168.56.10:9100'
        - '192.168.56.20:9100'
        - '192.168.56.30:9100'
```

После изменения конфигурации я проверял её:

promtool check config /etc/prometheus/prometheus.yml

Затем перезапускал Prometheus:

sudo systemctl restart prometheus

6.2. Проверка через метрику up

В Prometheus я использовал запрос:

up

Этот запрос оказался очень полезным.

Если Prometheus успешно получает метрики с определённого адреса, он показывает 1.

Если получить метрики не удалось, он показывает 0.

Например:

192.168.56.10:9100 = 1

192.168.56.20:9100 = 1

192.168.56.30:9100 = 1

Это означает, что Prometheus успешно получает данные со всех трёх Node Exporter.

При этом я узнал одну важную вещь: up = 0 не всегда означает, что сервер полностью выключен.

Возможно, сам сервер продолжает работать, но Node Exporter остановился. Или возникла проблема с сетью, портом либо соединением.

Поэтому только по одной метрике up нельзя точно определить причину неисправности.

7. Установка Grafana

Когда Prometheus начал получать данные, я перешёл к Grafana.

Prometheus может хранить метрики и выполнять запросы, но мне хотелось видеть всю инфраструктуру на одном экране в удобном виде.

Для этого я установил Grafana на MON-01.

После установки запустил её службу:

sudo systemctl enable --now grafana-server

Затем открыл веб-интерфейс:

http://192.168.56.10:3000

После входа в Grafana я добавил Prometheus в качестве источника данных.

Для подключения использовал:

http://localhost:9090

Такой адрес подходит, потому что Grafana и Prometheus работают на одном сервере MON-01.

После настройки Grafana получила возможность выполнять запросы к Prometheus и показывать результаты.

8. Самостоятельное создание первого дашборда

Следующим этапом было создание дашборда Bank Infrastructure Monitoring.

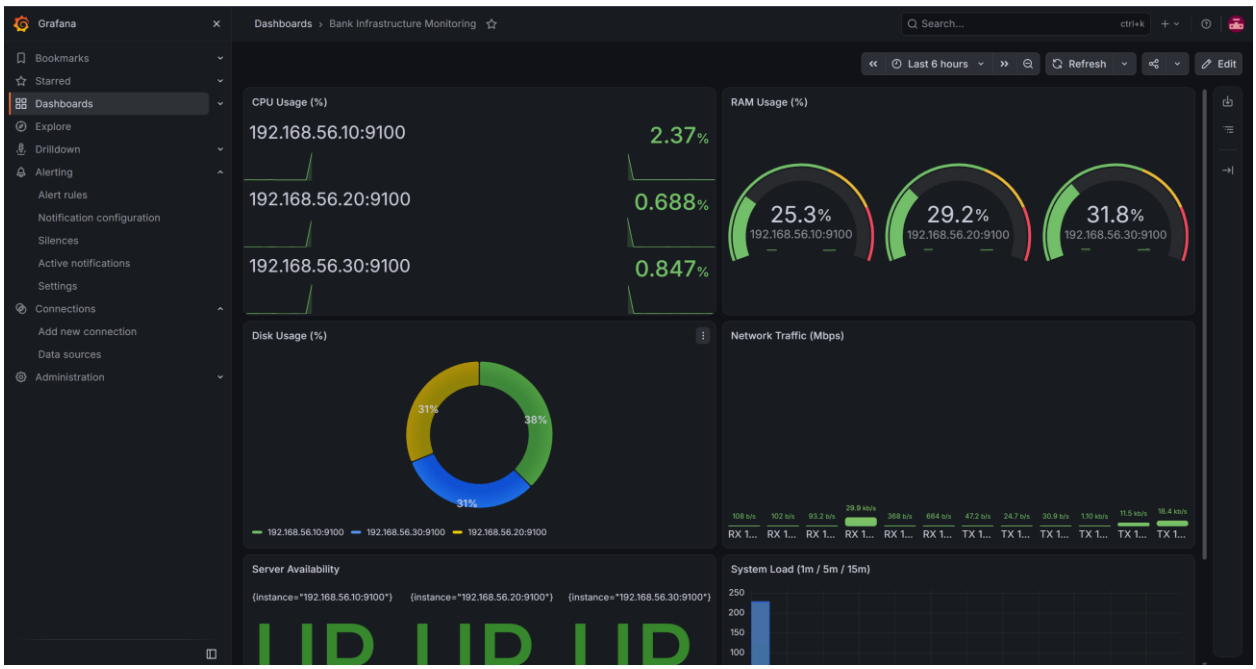
Я решил не использовать готовые дашборды, которые можно скачать и импортировать в Grafana.

Вместо этого я создавал панели вручную.

Для каждой панели мне нужно было выбрать тип отображения, написать PromQL-запрос, указать единицы измерения, настроить подписи и при необходимости цвета.

Сначала интерфейс Grafana был непривычным. Некоторые настройки находились не там, где я ожидал, поэтому приходилось открывать редактор панелей и разбираться в его возможностях.

Постепенно я научился создавать собственные визуализации.



8.1. Мониторинг процессора — CPU Usage

Первой важной панелью стал мониторинг процессора.

Для него я использовал PromQL-запрос:

```
100 * (
  1 - avg by(instance) (
    rate(node_cpu_seconds_total{mode="idle"}[5m])
  )
)
```

Вначале этот запрос выглядел для меня сложным.

Позже я разобрал его по частям.

node_cpu_seconds_total — метрика, которая показывает, сколько времени процессор проводил в разных состояниях.

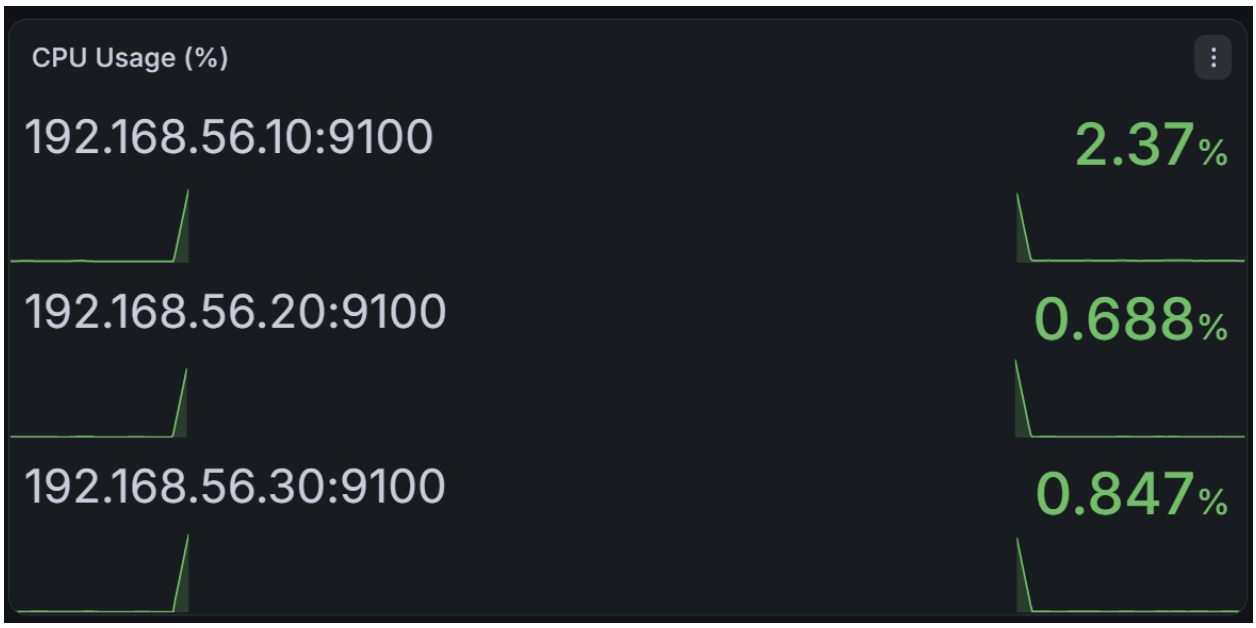
mode="idle" означает время простоя процессора.

rate(...[5m]) вычисляет среднюю скорость изменения счётчика за последние пять минут.

avg by(instance) объединяет показатели процессоров одного сервера.

А 1 - ... позволяет получить занятую часть процессорного времени вместо свободной.

Умножение на 100 переводит результат в проценты.



Например, если CPU Usage показывает 90%, это означает, что процессор в среднем был сильно загружен за выбранный период.

На панели я мог сравнивать загрузку трёх серверов.

8.2. Мониторинг оперативной памяти — RAM Usage

Следующей панелью стал мониторинг оперативной памяти.

Для расчёта использовал:

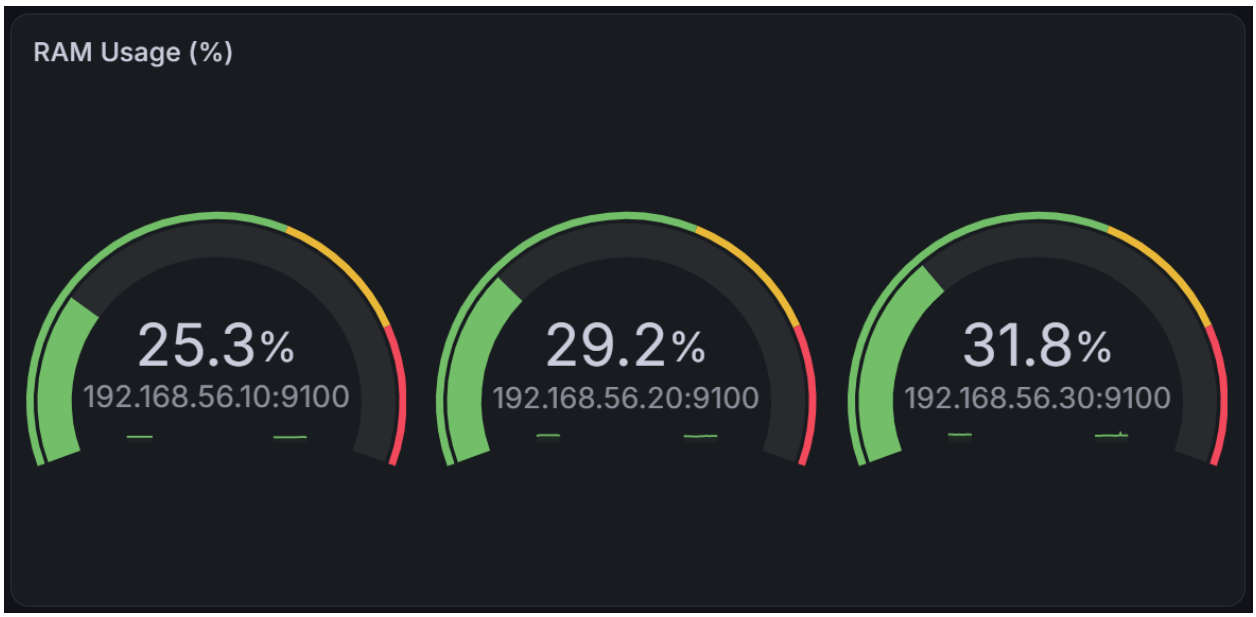
```
100 * (
  1 -
  node_memory_MemAvailable_bytes
  /
  node_memory_MemTotal_bytes
)
```

Здесь MemTotal показывает общий объём оперативной памяти.

MemAvailable показывает, сколько памяти система примерно может предоставить приложениям без существенного использования swap.

Во время изучения этой метрики я узнал, что Linux использует свободную оперативную память для кэширования данных.

Поэтому просто смотреть на MemFree не всегда правильно.



Например, если сервер имеет 8 ГБ RAM, а 6 ГБ занято, это не обязательно означает, что памяти скоро не хватит. Часть памяти может использоваться для кэша, который система при необходимости освободит.

Поэтому для оценки нехватки памяти я использовал MemAvailable.

8.3. Мониторинг диска — Disk Usage

После CPU и RAM я создал панель мониторинга дискового пространства.

Для этого использовал метрики:

node_filesystem_avail_bytes

node_filesystem_size_bytes

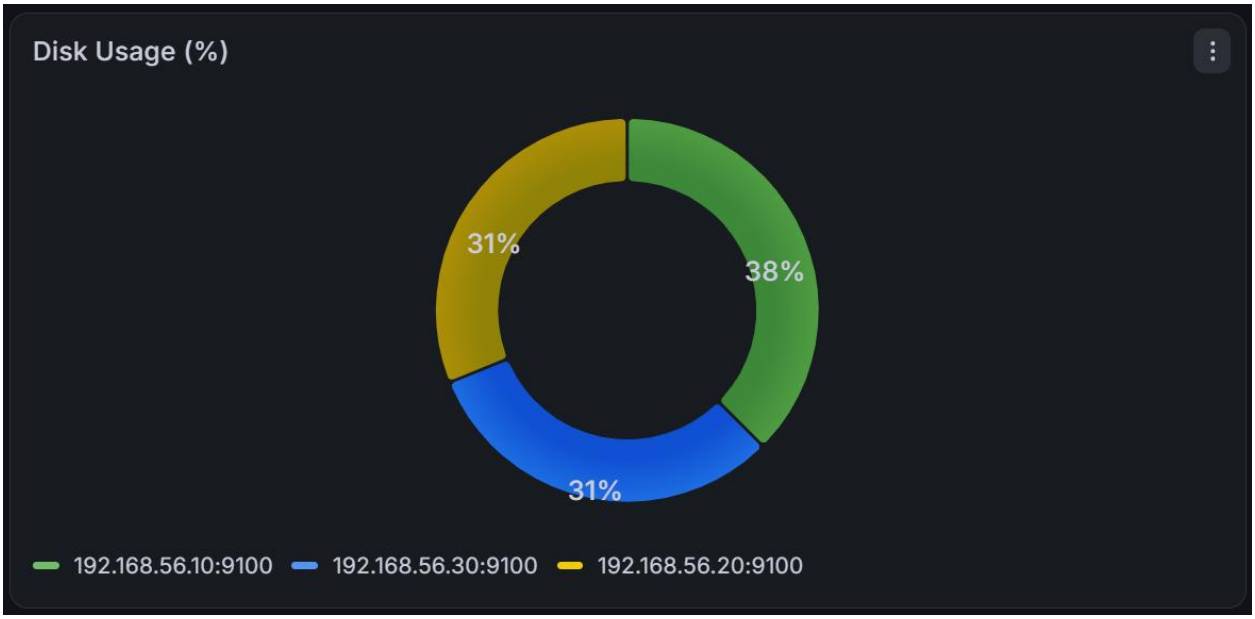
Первая показывает доступное дисковое пространство.

Вторая — общий размер файловой системы.

Для расчёта процента заполнения использовал:

```
100 * (
  1 -
  node_filesystem_avail_bytes{mountpoint="/"}
  /
  node_filesystem_size_bytes{mountpoint="/"}
)
```

Я понял, что мониторинг диска особенно важен для серверов баз данных.



Например, если PostgreSQL работает на сервере, где практически закончилось свободное место, новые данные могут перестать записываться.

Это может привести к ошибкам приложения и остановке отдельных операций.

При этом процессор и оперативная память могут оставаться в нормальном состоянии.

Также я узнал, что заполнение диска не обязательно означает повреждение данных, но может серьёзно повлиять на доступность базы данных.

8.4. Мониторинг сети — Network Traffic

Следующей панелью стал сетевой трафик.

Я хотел видеть, сколько данных сервер получает и отправляет.

Для входящего трафика использовал:

rate(node_network_receive_bytes_total[5m]) * 8 / 1000000

Для исходящего:

rate(node_network_transmit_bytes_total[5m]) * 8 / 1000000

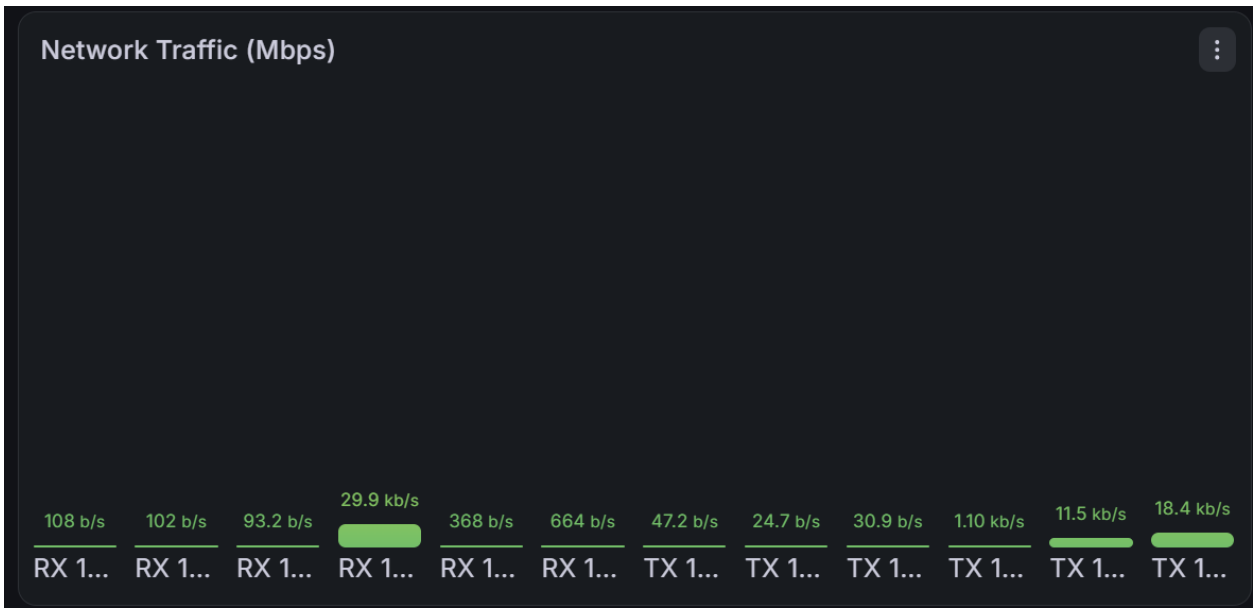
Здесь я разобрался с единицами измерения.

Node Exporter показывает количество байтов.

А скорость сети обычно удобно отображать в битах в секунду или мегабитах в секунду.

Поэтому значение нужно умножить на восемь, потому что один байт содержит восемь бит.

Затем результат делится на миллион, чтобы получить Мбит/с.



Например, если сервер передаёт 2 500 000 байт в секунду, это соответствует 20 Мбит/с.

Также я понял, что rate() нужен потому, что исходная метрика показывает накопленное количество переданных данных, а нам нужна именно скорость передачи.

8.5. Мониторинг System Load

Для мониторинга общей нагрузки Linux я добавил панель System Load.

Использовал три метрики:

node_load1

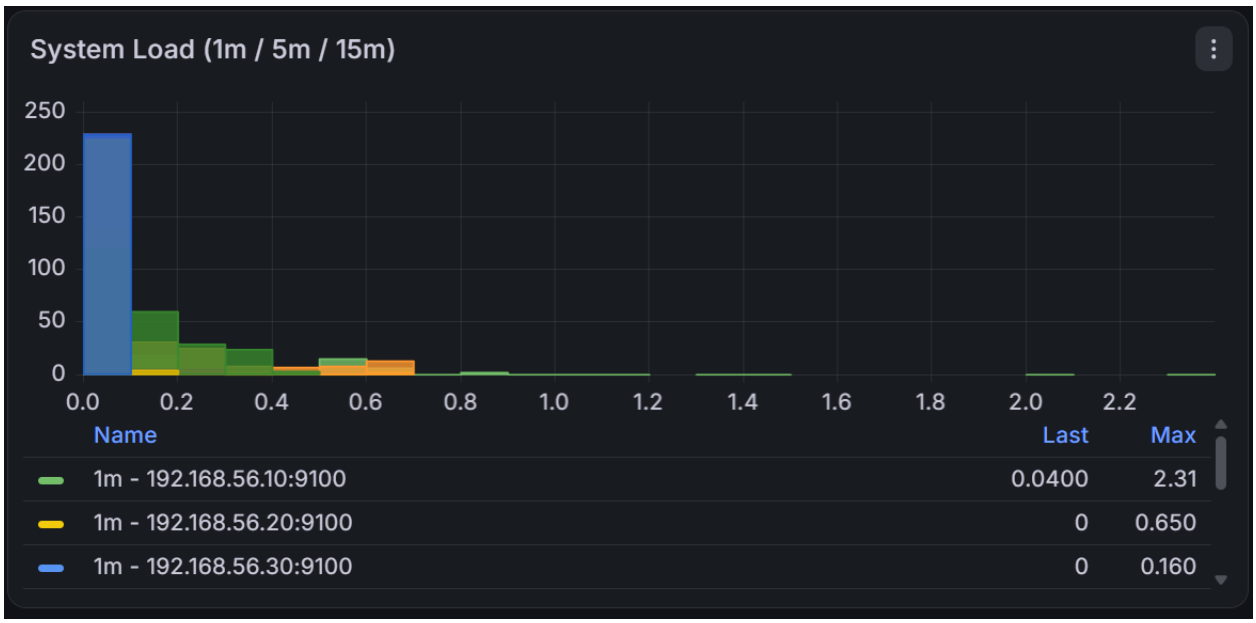
node_load5

node_load15

Они показывают среднюю нагрузку системы за одну, пять и пятнадцать минут.

Я узнал, что System Load — это не процент загрузки процессора.

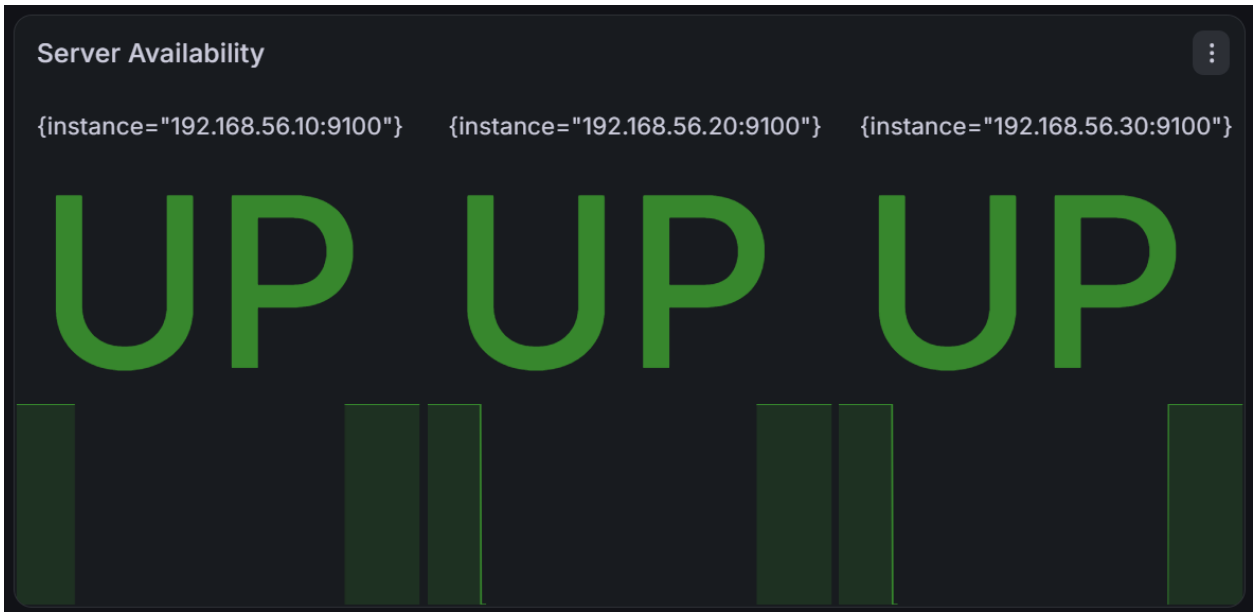
Этот показатель связан с количеством задач, которые выполняются, ожидают процессор или находятся в определённых состояниях ожидания ввода-вывода.



Чтобы правильно оценивать System Load, нужно учитывать количество логических процессоров сервера.

Например, Load Average 4 может означать разную ситуацию для сервера с двумя и сервера с восемью логическими процессорами.

8.6. Мониторинг доступности серверов



Последней из основных панелей стала Server Availability.

Для неё использовал:

up{instance=~"192\\.168\\.56\\.(10|20|30):9100"}

Я выбрал тип визуализации Stat.

Затем настроил отображение значений:

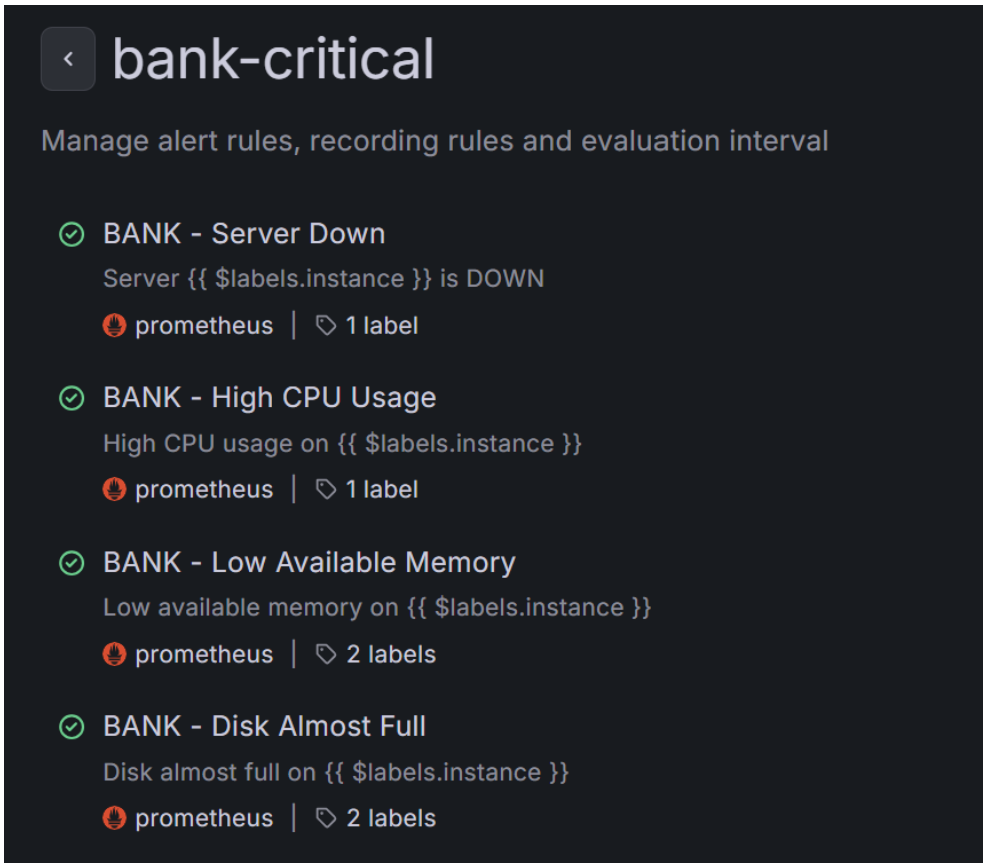
- 1 — UP.

- 0 — DOWN.

Для UP использовал зелёный цвет, для DOWN — красный.

Таким образом, можно было быстро увидеть, с какого сервера Prometheus перестал получать метрики.

9. Настройка уведомлений в Telegram



После создания дашборда я перешёл к настройке автоматических уведомлений.

Для этого я создал Telegram-бота через BotFather.

Затем в Grafana открыл раздел Alerting и создал Contact Point с типом Telegram.

Для подключения использовал токен бота и Chat ID.

9.1. Создание Server Down Alert

Первым алертом стал:

BANK - Server Down

Я использовал метрику up.

Настроил условие так, чтобы тревога возникала, если значение становится меньше единицы.

Также установил Pending period равным одной минуте.

Я узнал, что Pending period нужен для защиты от слишком частых и ложных уведомлений.

Например, если один запрос Prometheus не удался из-за кратковременной сетевой проблемы, не всегда нужно сразу отправлять тревогу.

Но если проблема сохраняется, Grafana переводит алерт в состояние Firing.

9.2. Настройка Reduce и Threshold

Во время настройки Alert Rule я столкнулся с тем, что в интерфейсе Grafana были отдельные выражения Reduce и Threshold.

Сначала я не понимал, где нужно выбирать Last, а где — Is below.

Позже разобрался:

Reduce с функцией Last берёт последнее значение метрики.

Threshold сравнивает это значение с заданным порогом.

Например, если мы проверяем up, то условие Is below 1 означает, что Prometheus не смог получить метрики.

Также я добавлял метки severity, чтобы обозначить важность инцидента.

9.3. Проверка настоящего срабатывания

После настройки я решил проверить, работает ли система.

Для этого подключился к APP-01 и остановил Node Exporter:

sudo systemctl stop prometheus-node-exporter

После этого Prometheus перестал получать метрики с APP-01.

Через некоторое время Grafana обнаружила проблему и отправила сообщение в Telegram.

Я получил уведомление о том, что сервер мониторинга метрик недоступен.

После этого снова запустил Node Exporter:

sudo systemctl start prometheus-node-exporter

Prometheus возобновил сбор метрик.

Grafana определила, что проблема исчезла.

Затем в Telegram пришло уведомление Resolved.

Правда, оно пришло с задержкой.

В результате я убедился, что система действительно обнаруживает остановку Node Exporter и сообщает о восстановлении.

10. Настройка дополнительных алертов

После Server Down я продолжил работу с другими правилами.

10.1. High CPU Usage

Я создал правило для высокой загрузки процессора.

Условие было следующим:

CPU Usage выше 90%.

Pending period — пять минут.

Я выбрал такой период, потому что процессор может кратковременно загружаться почти на 100%, например во время запуска программы.

Если нагрузка длится всего несколько секунд, это не всегда проблема.

Но если она сохраняется несколько минут, специалисту мониторинга стоит обратить на неё внимание.

Для создания искусственной нагрузки на APP-01 я использовал stress-ng.

Это позволило наблюдать, как увеличивается CPU Usage на графике.

10.2. Low Available Memory

Также я настроил правило для оперативной памяти.

Условие: доступно менее 10% RAM.

Pending period — три минуты.

Для этого использовал метрику MemAvailable.

Задача правила — заранее предупредить о нехватке памяти.

10.3. Disk Almost Full

Ещё одно правило предназначалось для контроля заполнения диска.

Условие: занято более 90% дискового пространства.

Pending period — две минуты.

Я понял, что этот алерт особенно важен для сервера базы данных, потому что нехватка свободного места может привести к отказу операций записи.

Отдельное полноценное тестирование уведомлений RAM и Disk Alert я не подтверждал, поэтому не могу утверждать, что проверил все возможные сценарии.

11. Установка PostgreSQL на DB-01

После настройки основного мониторинга я решил расширить проект.

На DB-01 установил PostgreSQL.

В моей лаборатории использовалась версия PostgreSQL 18.6.

Я проверял состояние службы:

sudo systemctl status postgresql

И проверял, какой порт прослушивает PostgreSQL:

sudo ss -lntp | grep 5432

В результате увидел, что PostgreSQL слушает адрес:

127.0.0.1:5432

Это означает, что подключение по этому адресу возможно только локально, с самого DB-01.

12. Установка PostgreSQL Exporter

После установки PostgreSQL я захотел получать метрики самой базы данных.

Для этого установил PostgreSQL Exporter:

sudo apt install prometheus-postgres-exporter -y

Он предоставляет метрики на порту 9187.

Я создал отдельного пользователя PostgreSQL:

postgres_exporter

И выдал ему роль:

GRANT pg_monitor TO postgres_exporter;

Эта роль позволяет получать информацию, необходимую для мониторинга.

Я не использовал суперпользователя PostgreSQL для постоянного подключения Exporter, потому что для мониторинга не нужны полные административные права.

Затем настроил подключение Exporter к PostgreSQL.

После настройки проверил метрику:

pg_up

Получил значение:

pg_up 1

Это означало, что PostgreSQL Exporter смог успешно подключиться к базе данных.

13. Подключение PostgreSQL Exporter к Prometheus

Следующим этапом я настроил получение метрик PostgreSQL на MON-01.

Сначала проверил настройки systemd-службы PostgreSQL Exporter.

Использовал:

systemctl cat prometheus-postgres-exporter

Из вывода узнал, что служба использует конфигурационный файл:

/etc/default/prometheus-postgres-exporter

И переменную ARGS для параметров запуска.

Для доступа по внутренней сети использовал адрес:

192.168.56.30:9187

Затем добавил PostgreSQL Exporter в конфигурацию Prometheus.

Пример:

```
- job_name: 'postgresql'
  static_configs:
    - targets: ['192.168.56.30:9187']
      labels:
        server: 'DB-01'
```

После этого Prometheus получил возможность собирать метрики PostgreSQL.

Я разобрался в различии между up и pg_up.

Если up = 1, это означает, что Prometheus успешно получает метрики с Exporter.

Если pg_up = 1, это означает, что Exporter успешно подключается к самой базе данных.

Например, возможна ситуация, когда PostgreSQL Exporter работает, но PostgreSQL недоступен.

В таком случае up может оставаться равным единице, а pg_up будет равен нулю.

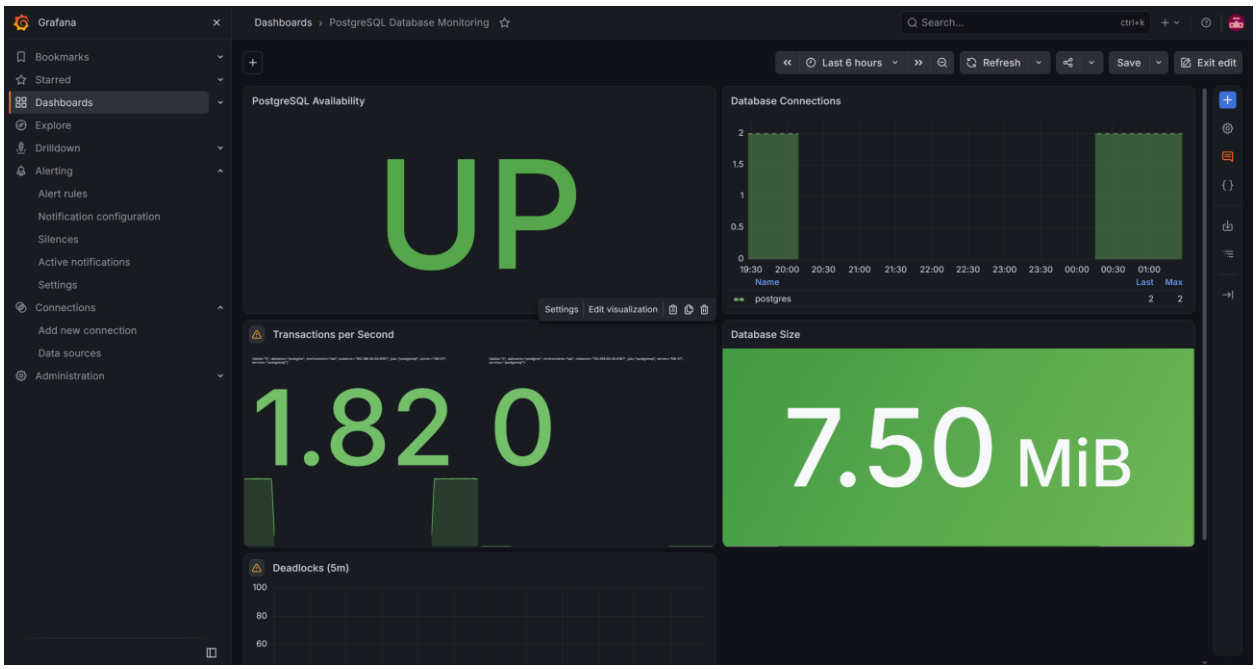
Это помогло мне понять, почему недостаточно контролировать только доступность операционной системы.

14. Создание PostgreSQL Dashboard

После подключения PostgreSQL Exporter я создал отдельный дашборд:

PostgreSQL Database Monitoring

В нём я разместил несколько панелей.



14.1. PostgreSQL Availability



Для этой панели использовал:

pg_up{job="postgresql"}

Она показывает, может ли Exporter подключиться к PostgreSQL.

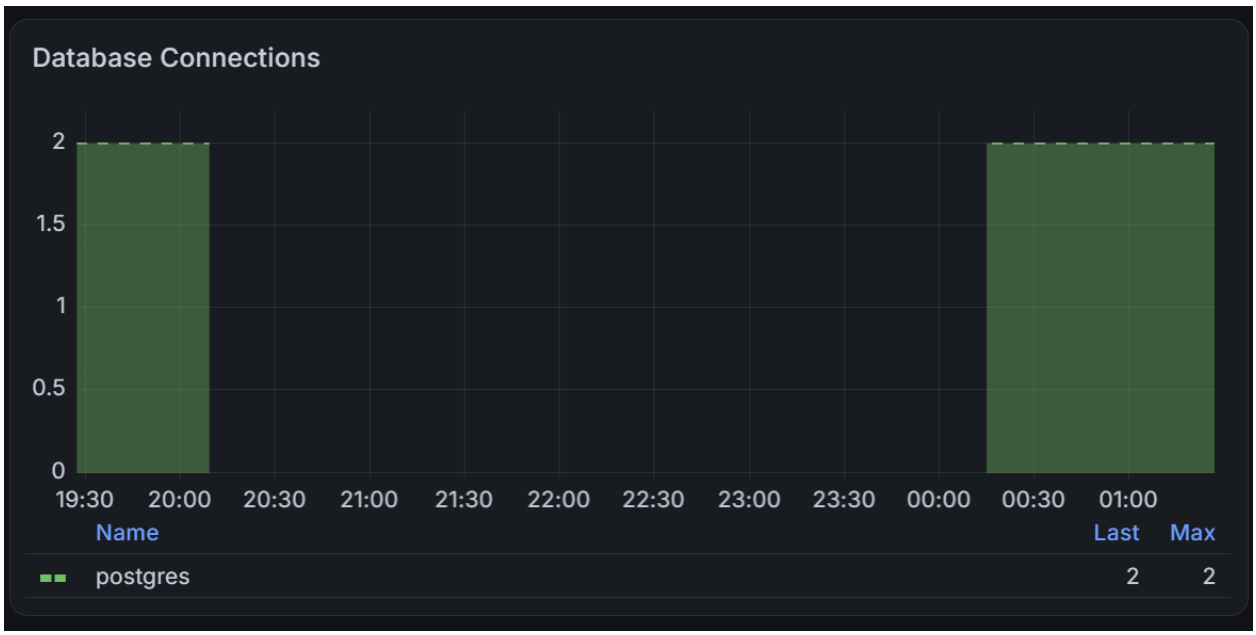
Если значение равно единице, подключение работает.

Если нулю — нужно искать причину проблемы.

14.2. Database Connections

Для мониторинга подключений использовал:

pg_stat_database_numbackends{datname="postgres"}



Эта метрика показывает количество текущих подключений к базе данных.

Я понял, что большое количество подключений может стать проблемой, даже если процессор и оперативная память загружены не сильно.

Например, если PostgreSQL разрешает максимум 100 подключений, а уже занято 95, новые подключения могут скоро начать отклоняться.

Поэтому количество соединений тоже нужно контролировать.

14.3. Transactions per Second



Следующей панелью стала Transactions per Second.

Она показывает, сколько транзакций PostgreSQL обрабатывает за секунду.

Для успешных транзакций использовал:

rate(pg_stat_database_xact_commit{datname="postgres"}[5m])

Для откатов:

rate(pg_stat_database_xact_rollback{datname="postgres"}[5m])

14.4. Database Size

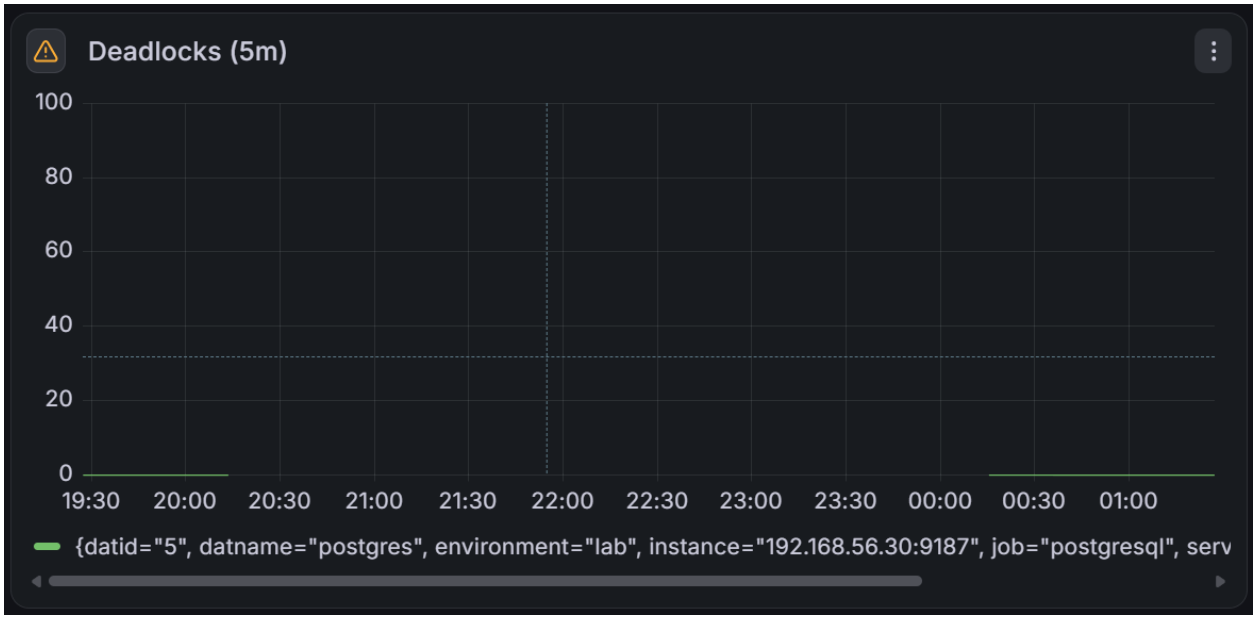


Для отображения размера базы данных использовал:

pg_database_size_bytes

В Grafana выбрал единицы измерения Bytes (IEC).

14.5. Deadlocks



Также создал панель Deadlocks.

Для неё использовал:

increase(pg_stat_database_deadlocks{datname="postgres"}[5m])

Deadlock — это ситуация, когда транзакции мешают друг другу продолжить работу из-за взаимных блокировок.

PostgreSQL обнаруживает такую ситуацию и прерывает одну из транзакций.

Панель показывает, насколько увеличилось количество deadlocks за последние пять минут.

15. Какие трудности у меня возникали

Во время проекта было несколько ситуаций, когда приходилось искать причину проблемы.

Проблемы с сетью. Сначала виртуальные машины не всегда были доступны. Я проверял настройки VMware, Host-only, NAT и сетевые адаптеры Windows.

Непривычный интерфейс Grafana. Некоторые настройки приходилось искать самостоятельно, потому что интерфейс отличался от инструкций.

Настройка Reduce и Threshold. Сначала я путал функции обработки значения и условия срабатывания алерта.

Задержка Telegram Resolved. После восстановления Node Exporter сообщение пришло не сразу. Я изучил, какие интервалы влияют на отправку уведомлений.

Эти трудности были частью обучения. Благодаря им я не просто повторял команды, а постепенно понимал, почему система работает именно так.

16. Что в итоге получилось

В результате у меня получилась учебная система мониторинга из трёх виртуальных серверов.

На MON-01 работают Prometheus и Grafana.

На всех трёх серверах работает Node Exporter.

На DB-01 дополнительно работают PostgreSQL и PostgreSQL Exporter.

Prometheus собирает метрики, а Grafana показывает их в виде графиков и отдельных панелей.

Я самостоятельно создал основной дашборд с мониторингом процессора, оперативной памяти, дискового пространства, сетевого трафика, System Load и доступности серверов.

Также создал отдельный дашборд PostgreSQL с показателями доступности базы данных, подключений, транзакций, размера базы и deadlocks.

Для уведомлений использовал Telegram.

17. Чему я научился

Я научился использовать Prometheus для сбора метрик с нескольких серверов.

Разобрался с основными метриками CPU, RAM, дисков и сети.

Научился самостоятельно создавать дашборды в Grafana и настраивать их внешний вид.

Получил опыт создания Alert Rules, настройки условий, времени ожидания и отправки уведомлений в Telegram.

Также познакомился с мониторингом PostgreSQL и понял, что доступность самого сервера не означает, что все его сервисы работают правильно.

Самое важное для меня — я начал лучше понимать, как искать причину проблемы, а не просто смотреть на красные и зелёные показатели.

18. Заключение

Этот проект стал для меня практической работой, в которой я смог объединить знания Linux, сетей, мониторинга.

Я начал с создания трёх виртуальных машин, затем настроил между ними сеть, установил Node Exporter, Prometheus и Grafana, самостоятельно создал дашборды и настроил уведомления в Telegram.

После выполнения основной части я расширил проект, добавив PostgreSQL и PostgreSQL Exporter.

Не все этапы проходили с первого раза. Были проблемы с сетью, настройками Grafana, PromQL и подключением Exporter. Но именно благодаря этим проблемам я получил больше практического опыта.

В итоге я выполнил основную часть задания по мониторингу серверов и создал работающий учебный стенд.

При этом я понимаю, что мониторинг — это не только установка программ и создание графиков. Нужно уметь правильно читать метрики, отличать сбой сервера от сбоя отдельного сервиса, оценивать серьёзность проблемы и своевременно сообщать о ней ответственным специалистам.

В дальнейшем я хочу учиться более подробно анализировать инциденты и дальше развиваться в сфере ИБ.